

Technical Summary:

Monte Carlo Tree Search Boosts Reasoning via Iterative Preference Learning

Kai-Yu Lu

1 Research Problem and Motivation

Multi-step reasoning with Large Language Models (LLMs) is limited by sparse learning signals when supervision is primarily tied to final-answer correctness. Instance-level preference learning labels entire responses as preferred or dispreferred, which provides limited guidance for identifying which intermediate reasoning steps cause success or failure. This work targets the credit-assignment gap by constructing step-level preference supervision.

A second motivation is that iterative self-improvement depends on collecting preference data aligned with the current policy distribution. Preference learning performed on static, offline data can become mismatched as the policy changes, reducing the effectiveness of continued improvement. The work therefore emphasizes an on-policy, iterative protocol that repeatedly collects new preferences using the latest policy.

A third motivation is practical: search-based reasoning provides look-ahead improvements but requires a critic signal. Training separate reward or value networks increases complexity. The work combines outcome-based validation with stepwise self-evaluation to supply a critic signal directly during search.

2 Related Work

Preference-based alignment. Reinforcement Learning from Human Feedback (RLHF) commonly trains a reward model from pairwise preferences and then optimizes the policy with reinforcement learning. Direct Preference Optimization (DPO) updates the policy directly from preferences without explicitly training a reward model, improving stability and scalability. This work adopts a DPO-style update but changes the preference collection protocol by generating preferences through search.

Iterative self-training and self-improvement. Prior iterative methods generate candidate solutions or rationales, filter them using heuristics or self-consistency, and fine-tune on selected outputs. These approaches usually treat the entire response as the unit of supervision, limiting process-level credit assignment. This work differs by generating step-level preferences using look-ahead search and optimizing the policy on these fine-grained signals.

Search for LLM reasoning. Test-time sampling methods such as self-consistency improve reasoning by drawing multiple solutions and aggregating. Search-based methods can provide stronger exploration but require a scoring mechanism. This work uses Monte Carlo Tree Search (MCTS) to construct preference labels for training, rather than only improving inference-time decoding.

Process supervision and stepwise evaluation. Process-level supervision and stepwise evaluation can improve reasoning by guiding intermediate steps. This work operationalizes process guidance by decomposing instance-level signals into step-level preferences derived from MCTS values.

3 Dataset Construction

The evaluation spans arithmetic reasoning and commonsense multiple-choice reasoning. A supervised fine-tuning dataset is used to initialize the policy, and two prompt pools are constructed for iterative preference collection.

Datasets and Roles

Table 1: Datasets and their roles. Missing fields are marked as “Not specified in the paper.”

Dataset	Task Type	Description and Usage in This Work	Size / Split
Arithmo	Arithmetic, coding	Supervised fine-tuning (SFT) dataset used to obtain an initial policy; described as approximately 540K math and coding problems.	≈540K problems
GSM8K	Arithmetic word problems	Evaluated for chain-of-thought and program-of-thought reasoning; used to construct a prompt dataset for preference learning.	Not specified in the paper.
MATH	Competition math	Evaluated for challenging math reasoning; used to construct a prompt dataset for preference learning; ground-truth reasoning steps are used in a reported configuration.	Not specified in the paper.
ARC (Easy/Challenge)	Multiple-choice science	Included in the commonsense training mixture and evaluated; ARC-C refers to the challenge split.	Not specified in the paper.
AI2Science (Elem/Middle)	Multiple-choice science	Included in the commonsense training mixture and evaluated; the middle split is reported.	Not specified in the paper.
OpenBookQA (OBQA)	Multiple-choice science	Included in the training mixture and discussed as less efficient for intermediate reasoning refinement than direct tuning.	Not specified in the paper.
CommonSenseQA (CSQA)	Multiple-choice commonsense	Included in the training mixture and evaluated.	Not specified in the paper.
SciQ	Multiple-choice science	Unseen evaluation dataset used to test generalization.	Not specified in the paper.

Prompt Pool Construction for Preference Learning

Table 2: Prompt pools used for MCTS-based preference collection.

Pool	Source Datasets	Construction and Usage	Size
Arithmetic prompts	GSM8K + MATH (train)	Combined training prompts used for step-level preference collection via MCTS during iterative learning.	24K prompts
Commonsense prompts	ARC + AI2Science + OBQA + CSQA (train)	Combined training prompts used for step-level preference collection via MCTS during iterative learning.	12K prompts

4 Query Protocol and Task Definitions

Input and Output Format

Each instance is a prompt x (a question). The model outputs a reasoning chain and a final answer. For multiple-choice tasks, the final answer corresponds to a candidate option.

Step Decomposition and Preference Labels

The response is decomposed into discrete steps. A *state* s_t is the current prefix of the reasoning chain at step t . An *action* a is a candidate next reasoning step (a token sequence) appended to the prefix, producing a child state s_{t+1} .

For each depth t in the search tree, sibling candidate next steps are compared using MCTS-derived action values. The highest-valued candidate defines the preferred step response y_w , and the lowest-valued candidate defines the dispreferred step response y_l . The training data consists of tuples (x, y_w, y_l) .

Success Criterion and Metrics

The primary success criterion is **accuracy** (percentage of correct final answers) on each dataset validation set. Additional analyses compare training-time compute scaling across iterative checkpoints with test-time compute scaling via self-consistency (SC@ k) sampling and cumulative checkpoint aggregation.

5 Modeling Approach

The method alternates between search-based preference collection and preference-based policy optimization.

Iterative Loop

Step 1: Initialize an SFT policy. A base model (Mistral-7B) is supervised fine-tuned on Arithmo to obtain an initial policy π_{sft} .

Step 2: Collect step-level preferences with MCTS. For each prompt, MCTS explores candidate next steps across multiple depths, evaluates partial reasoning using a combined critic signal, and assigns action values to candidate transitions. Step-level preference pairs are extracted by taking the best and worst sibling candidates at each depth.

Step 3: Update the policy with DPO and label smoothing. The collected step-level preferences are used to optimize a DPO-style objective, producing an updated policy. Preference collection for iteration i is performed using the policy from iteration $i - 1$ (on-policy protocol).

Core Equations (Core-Only, Purpose-Driven)

Four equations are included because the core mechanism depends on the PUCT selection rule, the reward construction for partial reasoning, the conservative DPO objective with adaptive label smoothing, and the visit-count-derived smoothing rule.

(1) PUCT selection rule.

$$s_{t+1}^* = \arg \max_{s_t} \left[Q(s_t, a) + c_{\text{puct}} \cdot p(a \mid s_t) \cdot \frac{\sqrt{N(s_t)}}{1 + N(s_{t+1})} \right] \quad (1)$$

Purpose: Selects which child to traverse during MCTS, balancing exploitation and exploration for preference data collection.

Symbols: s_t is the current state (reasoning prefix) at step t ; a is a candidate next step; s_{t+1} is the resulting child state; $Q(s_t, a)$ is an estimated action value; $N(s_t)$ and $N(s_{t+1})$ are visit counts; c_{puct} is a search coefficient; $p(a \mid s_t)$ is an action prior. The prior is defined as $p(a \mid s_t) = \pi_\theta(a \mid x, s_t) / |a|^\lambda$, where λ applies a length penalty.

Intuition: Q prefers steps that lead to better downstream outcomes, while the exploration bonus favors branches that have not been visited frequently. The length penalty prevents excessively long steps from dominating the prior.

Usage: Used in the selection phase of MCTS for traversing from the root to a leaf before expansion.

(2) State reward for partial reasoning.

$$R(s_t) = O(s_t) + C(s_t) \quad (2)$$

Purpose: Constructs a critic signal for intermediate states by combining terminal outcome validation with dense stepwise self-evaluation.

Symbols: $R(s_t)$ is the reward score; $O(s_t)$ is outcome correctness, defined as 1 for correct terminal states, -1 for incorrect terminal states, and 0 for unfinished states; $C(s_t)$ is a self-evaluation score for the partial reasoning prefix.

Intuition: Outcome-only scoring is sparse and provides no learning signal for intermediate steps. Adding self-evaluation supplies dense guidance that can steer search and produce informative preferences.

Usage: Computed during expansion and rollout evaluation in MCTS to score states and support value backups.

(3) Conservative DPO objective with label smoothing.

$$\ell_i(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim D_i} \left[(1 - \alpha_{x, y_w, y_l}) \log \sigma(\beta h_{\pi_\theta}^{y_w, y_l}) + \alpha_{x, y_w, y_l} \log \sigma(-\beta h_{\pi_\theta}^{y_w, y_l}) \right] \quad (3)$$

Purpose: Updates the policy on step-level preferences collected at iteration i while reducing sensitivity to noisy preference labels.

Symbols: D_i is the preference dataset at iteration i ; x is the prompt; y_w and y_l are preferred and dispreferred step responses; $\sigma(\cdot)$ is the sigmoid; β scales the objective; $h_{\pi_\theta}^{y_w, y_l}$ is defined in the paper as a log-ratio difference comparing the current policy π_θ and a reference policy π_{ref} on y_w versus y_l ; α_{x, y_w, y_l} is a smoothing weight.

Intuition: The objective increases the relative probability of preferred steps and decreases that of dispreferred steps, while the smoothing term makes updates conservative when the preference label is uncertain.

Usage: Minimized after MCTS data collection to produce the updated policy used in the next iteration.

(4) Adaptive label smoothing from visit counts.

$$\alpha_{x, y_w, y_l} = \frac{1}{N(x, y_w)/N(x, y_l) + 1} \quad (4)$$

Purpose: Determines the smoothing weight using the confidence implied by MCTS visitation statistics.

Symbols: $N(x, y_w)$ and $N(x, y_l)$ are visit counts associated with the preferred and dispreferred step candidates under prompt x .

Intuition: A larger visit ratio indicates stronger search preference for y_w , producing smaller α and a more decisive update. Similar visit counts indicate uncertainty, producing larger α and a more conservative update.

Usage: Computed per preference pair and plugged into Equation (3).

Minimal Example (Search to Preference to Update)

Given a prompt x , at a partial reasoning state s_t , the policy proposes several candidate next steps. MCTS evaluates each candidate by exploring short continuations and scoring resulting states using $R(s_t)$ from Equation (2). At depth t , the candidate with the highest action value defines y_w and the lowest defines y_l . Repeating over depths yields step-level preference tuples (x, y_w, y_l) . The policy is then updated by minimizing the DPO loss in Equation (3), which increases the probability of y_w relative to y_l under the updated policy.

6 Empirical Results

Training and Evaluation Setup

Table 3: Training, preference learning, and MCTS configuration. Missing fields are marked as “Not specified in the paper.”

Component	Configuration
Base model	Mistral-7B
SFT optimizer and schedule	Learning rate $5 \cdot 10^{-6}$ with cosine scheduler
SFT batch size	128
SFT max sequence length	512
Preference learning method	Direct Preference Optimization (DPO)
DPO learning rate	10^{-6}
DPO batch size	32
DPO β	0.1
Max step length (for step candidates)	64
MCTS iterations	$K = 5$ (all tasks)
Arithmetic MCTS breadth and depth	$b_1 = 5$ annealed to $b_2 = 3$, depth $d = 4$
Commonsense MCTS breadth and depth	$b_1 = 4$ annealed to $b_2 = 2$, depth $d = 3$
Prompt pools used for MCTS	24K arithmetic prompts; 12K commonsense prompts
Hardware	Up to 4×40GB NVIDIA A100 GPUs
Training cost (reported)	Arithmetic collection about 2 minutes per prompt; about 30 A100 days per epoch
Early stopping (reported)	Around 30% training data for arithmetic; around 50% for commonsense
Other hyperparameters	Not specified in the paper.

Metrics

Table 4: Evaluation metrics and their meanings.

Metric	Definition and Usage
Accuracy (%)	Percentage of instances with correct final answers on the validation set. Primary metric across tasks.
Pass rate / cumulative pass rate	Used in compute scaling analyses across iterative checkpoints; cumulative versions aggregate across checkpoints.
Self-consistency (SC@ k) accuracy	Sampling-only test-time scaling by generating k solutions and aggregating by consistency.
AUC and self-evaluation accuracy	Quantify the discriminative ability of the self-evaluation signal used as part of the critic during MCTS.

Main Results: Arithmetic Reasoning

Table 5: Arithmetic reasoning results (accuracy %).

Method (Base: Mistral-7B)	GSM8K	MATH
SFT (Arithmo)	75.9	28.9
MCTS Offline-DPO	79.9	31.9
Instance-level Online-DPO	79.7	32.9
Step-level Online-DPO	80.7	32.2
Step-level Online-DPO (with ground truth)	81.8	34.7

Interpretation. Step-level online preference learning improves GSM8K from 75.9 to 80.7 (+4.8 points), and the configuration using ground-truth guidance improves GSM8K to 81.8 (+5.9 points). On MATH, the best reported result is 34.7 versus 28.9 for SFT (+5.8 points). The offline variant is weaker than the best online configuration (79.9 vs 81.8 on GSM8K; 31.9 vs 34.7 on MATH), consistent with the emphasis on on-policy preference collection. Instance-level online training is competitive on MATH (32.9) but does not match the best step-level result (34.7).

Main Results: Commonsense Reasoning

Table 6: Commonsense reasoning results (accuracy %). Train Data Used (%) is reported where provided.

Method (Base: Mistral-7B)	ARC-C	AI2Sci-M	CSQA	SciQ	Train Data Used (%)
SFT Base (Arithmo)	60.6	70.9	54.1	80.8	Not specified in the paper.
Direct Tuning	73.9	85.2	79.3	86.4	100
MCTS Offline-DPO	70.8	82.6	68.5	87.4	19.2
Instance-level Online-DPO	75.3	87.3	63.1	87.6	45.6
Step-level Online-DPO	76.4	88.2	74.8	88.5	47.8

Interpretation. Relative to direct tuning, step-level online preference learning improves ARC-C (76.4 vs 73.9), AI2Sci-M (88.2 vs 85.2), and SciQ (88.5 vs 86.4). The reported Train Data Used indicates that step-level online training reaches these results using 47.8% of the training data under the early stopping protocol, while direct tuning uses 100%. The offline variant is substantially weaker on ARC-C (70.8 vs 76.4) and AI2Sci-M (82.6 vs 88.2), highlighting the role of on-policy preference collection.

Observed weakness. The discussion states that intermediate reasoning refinement is less efficient than direct tuning for OBQA and CSQA. In the reported table, CSQA accuracy under step-level online training (74.8) is below direct tuning (79.3), indicating a task-dependent limitation under the reported setup.

Ablation: Self-Evaluation Prompt Design

Table 7: Ablation on including an “EXAMPLE ANSWER” in the self-evaluation prompt (AUC and accuracy %).

Approach	GSM8K		MATH		ARC-C	
	AUC	Acc.	AUC	Acc.	AUC	Acc.
With example answer	74.7	72.5	76.6	48.8	65.2	57.5
Without example answer	62.0	69.5	48.1	42.3	55.8	48.4

Interpretation. Including an example answer improves both AUC and accuracy across datasets, with a

large AUC increase on MATH (76.6 vs 48.1). This supports the reliance on self-evaluation as a component of the critic signal for search.

Training-Time vs Test-Time Compute Scaling

SciQ. The reported compute scaling analysis indicates that self-consistency plateaus around 84% on SciQ, while training-time iterative preference learning reaches about 88.6% pass rate, indicating stronger gains from training-time compute scaling on this unseen dataset.

MATH. The reported curves show that sampling-based test-time scaling exceeds the cumulative performance of iterative training, while iterative training improves pass@1 relative to the SFT baseline (for example, pass@1 increases from 29.0 to 32.2 at a checkpoint). This indicates a task-dependent trade-off between training-time preference learning and test-time sampling.

7 Summary

Key Innovations Relative to Prior Work

Step-level preference construction via MCTS. The method uses MCTS look-ahead to score candidate next steps and extract step-level preference pairs, replacing instance-level preference labels with process-level supervision.

Critic signal without training a separate reward model. Partial reasoning is scored by combining outcome correctness and stepwise self-evaluation, enabling search-based preference collection without additional reward or value networks.

On-policy iterative preference learning. Preferences are collected using the latest policy at each iteration, aligning data collection with the evolving policy distribution and addressing an offline failure mode discussed in the paper.

Noise-aware preference optimization. The DPO loss incorporates adaptive label smoothing based on visit counts, making preference updates conservative when MCTS confidence is low.

Limitations and Future Directions (Paper-Grounded)

Training-time compute cost. Step-level preference collection is expensive, reported as about 2 minutes per arithmetic prompt and about 30 A100 days per epoch, with early stopping used to reduce cost.

Optimization stability under online updates. The paper states that insufficient minimization of the DPO loss per iteration can lead to cyclic behavior in the online setting, motivating multiple optimizer steps per iteration.

Reduced efficiency on knowledge-heavy tasks. The paper reports that intermediate reasoning refinement is less efficient than direct tuning on OBQA and CSQA, and the main table shows CSQA under step-level online training remains below direct tuning.

Stated future direction. The paper proposes exploring MCTS hyperparameters (breadth, depth, number of steps, and iteration time) to study cost-performance trade-offs and discusses integrating trained reward model signals into the MCTS stage.